

KeenPay

Agentic commerce with bounded AI



Guardrails and safety

Guardrails and safety model

01 September 2026

<https://github.com/ambitiouss22/KeenPay>

Guardrails and safety

Every money action must be explainable, bounded, and recoverable. Policy runs in Python — not in the model.

Implementation target: `api/policy/engine.py`, `api/policy/anomaly.py`.

Trust layers

UNTRUSTED	user text, LLM output, inbound webhooks (verify first)
SEMI-TRUSTED	LangGraph – proposes <code>ProposedOffer</code> , never pays
TRUSTED	<code>PolicyEngine</code> , <code>compute_totals</code> , gated Razorpay client

Layer	Change price?	Call Razorpay?
User chat	no	no
LLM / agent	propose only	no
Policy engine	approve/clamp	no
<code>compute_totals</code>	deterministic	no
<code>create_payment_link</code>	no	yes (gated)
Webhook handler	status only	n/a

What counts as a money action

- 1 Setting `negotiated_unit_price_paise` or `discount_pct`
- 2 Setting `final_amount_paise`
- 3 Reserve / release inventory
- 4 Create / cancel Razorpay Payment Link
- 5 Mark order `paid` or `refunded`

Each needs: `decision_id` (when guardrail-related), `audit_logs` row, trace event.

Merchant policy (defaults)

```

class MerchantPolicy(BaseModel):
    policy_version: str = "2026.08.1"
    merchant_id: str
    currency: Literal["INR"] = "INR"
    max_discount_pct: float = 15.0
    max_discount_pct_per_sku: dict[str, float] = {}
    max_absolute_discount_paise: int = 50_000 # Rs 500
    min_margin_pct: float = 20.0
    cost_basis_field: Literal["cost_paise", "wholesale_paise"] = "cost_paise"
    max_qty_per_line: int = 10
    max_qty_per_order: int = 20
    allow_backorder: bool = False
    max_negotiation_rounds: int = 5
    max_payment_links_per_session_per_hour: int = 3
    block_on_anomaly_score_gte: float = 0.85

```

Python

Override at runtime via `MERCHANT_POLICY_JSON`.

Rules (evaluation order)

Sequential evaluation. Security anomaly can short-circuit to ESCALATED.

Rule ID	What it checks	On fail
<code>RULE_SECURITY_ANOMALY</code>	composite score	ESCALATED if ≥ 0.85
<code>RULE_PROMPT_INJECTION</code>	regex on user text	REJECTED
<code>RULE_MAX_DISCOUNT</code>	% cap	CLAMP to max
<code>RULE_MAX_ABSOLUTE_DISCOUNT</code>	paise cap	CLAMP
<code>RULE_MIN_MARGIN</code>	vs <code>cost_paise</code>	REJECTED
<code>RULE_INVENTORY_AVAILABLE</code>	on_hand - reserved	REJECTED
<code>RULE_INVENTORY_BOUNDS</code>	qty limits	REJECTED
<code>RULE_PRICE_SANITY</code>	> 0 integer paise	REJECTED
<code>RULE_OFFER_VERSION</code>	monotonic version	REJECTED
<code>RULE_CURRENCY</code>	INR only v1	REJECTED
<code>RULE_NEGOTIATION_ROUNDS</code>	$\leq$ max	ESCALATED at limit

RULE_MAX_DISCOUNT (sketch)

```
def rule_max_discount(offer, policy) -> RuleResult:
    cap = policy.max_discount_pct
    for item in offer.line_items:
        cap = min(cap, policy.max_discount_pct_per_sku.get(item.sku, cap))
    if offer.discount_pct <= cap:
        return RuleResult(passed=True, rule_id="RULE_MAX_DISCOUNT")
    return RuleResult(
        passed=False, rule_id="RULE_MAX_DISCOUNT", action="CLAMP",
        message=f"Discount capped at {cap}%",
        adjusted_offer=recalculate_offer(offer, discount_pct=cap),
    )
```

Python

Aggregation

```
def aggregate_results(results) -> GuardrailOutcome:
    if any(r.rule_id == "RULE_SECURITY_ANOMALY" and not r.passed for r in results):
        return "ESCALATED"
    if any(r.action == "REJECT" for r in results):
        return "REJECTED"
    if any(r.action == "CLAMP" for r in results):
        return "APPROVED" # after recalc on clamped offer
    return "APPROVED"
```

Python

Rate limits (Redis)

Key	Limit	Window
<code>ratelimit:chat:{user_id}</code>	30	60s
<code>ratelimit:guardrail:{session_id}</code>	60	60s
<code>ratelimit:payment_link:{session_id}</code>	3	3600s
<code>ratelimit:webhook:{ip}</code>	100	60s

On exceed: audit `RATE_LIMITED`, WS error, no money action.

Prompt injection

`api/policy/anomaly.py`:

```
INJECTION_PATTERNS = [
    r"(?i)ignore\s+(all\s+)?(previous|prior)\s+instructions",
    r"(?i)disregard\s+(policy|rules|guardrails)",
    r"(?i)create\s+(a\s+)?payment\s+link\s+for\s+Rs?\.\?1\b",
    r"(?i)system\s+prompt",
    r"(?i)bypass\s+(security|validation)",
    r"(?i)admin\s+override",
]
```

Python

Critical matches (price manipulation) set `security_block=True`, audit `SECURITY_PROMPT_INJECTION`, trace `security.flag`.

Anomaly score (v1 heuristic)

Signal	Weight
Injection flag	+0.5 each, cap 1.0
Discount ask > 2x max	+0.3
>3 offers in 30s	+0.2
"lowest without manager" phrasing	+0.25
Unicode tricks	+0.4

Score >= `block_on_anomaly_score_gte` -> ESCALATED.

Failure matrix

Scenario	Detection	Auto response	User sees	Audit
Prompt injection	regex + score	block, security_block	polite refusal	SECURITY_PROMPT_INJECTION
Margin violation	RULE_MIN_MARGIN	reject	counter within policy	GUARDRAIL_REJECTED
Discount > max	RULE_MAX_DISCOUNT	clamp	best authorized %	GUARDRAIL_CLAMPED
No stock	RULE_INVENTORY_AVAILABLE	reject	qty available	GUARDRAIL_REJECTED
Stale stock	DB < Redis hold	revalidate	refreshing offer	INVENTORY_REVALIDATION
Duplicate link request	idempotency hit	return cached URL	same link	PAYMENT_LINK_IDEMPOTENT
Webhook wrong amount	amount mismatch	payment_disputed	internal	WEBHOOK_AMOUNT_MISMATCH
Webhook replay	duplicate event_id	200 no-op	—	WEBHOOK_DUPLICATE
Bad SKU	not in catalog	skip line	product not found	CATALOG_SKU_INVALID
LLM timeout	10s	no state change	please wait	LLM_TIMEOUT
Razorpay 503	HTTP 503	retry 3x	link unavailable	PAYMENT_LINK_FAILED
Max rounds	round >= 5	HITL	specialist review	ESCALATED_MAX_ROUNDS
Rate limit	Redis	block	wait	RATE_LIMITED

Scenario	Detection	Auto response	User sees	Audit
Hold race	WATCH fail	retry once	transparent	INVENTORY_HOLD_CONFLICT
Engine crash	exception	ESCALATED	manual review	GUARDRAIL_ENGINE_ERROR

Degradation levels (`GET /api/v1/health`)

```

0 full
1 LLM down -> templates + search only
2 Redis down -> PG holds, tighter limits
3 Razorpay down -> save cart, no new links
4 Postgres down -> 503

```

Level ≥ 3 blocks new payment links.

Human-in-the-loop

Escalation -> `escalation_tickets` table.

Trigger	Priority
Anomaly score ≥ 0.85	P0
Policy engine error	P0
Payment disputed	P0
Max negotiation rounds	P1
User asks for human	P1

Human override rules:

Override	Limit	Role
+5% discount once	per session	support_agent
+10% discount	—	manager
Below margin floor	never in v1	—
Pay without guardrail	never	—

Override writes new `decision_id` with `actor=human`.

Inventory holds

```

1. APPROVED guardrail -> optional soft Redis hold
2. User confirms -> PG transaction:
   SELECT ... FOR UPDATE on products
   increment quantity_reserved, insert inventory_holds
3. Link expiry / cancel -> release hold
4. payment captured -> decrement on_hand, clear hold

```

Redis `hold:{session_id}:{sku}` mirrors PG for fast reads.

Payment gates

Before POST `https://api.razorpay.com/v1/payment_links`:

```
def assert_payment_gates(state, policy) -> None:
    assert state["guardrail_decision"] == "APPROVED"
    assert state["guardrail_decision_id"]
    assert state["user_confirmed_payment"] is True
    assert state["user_confirmed_at"]
    assert state["final_amount_paise"] > 0
    assert state["inventory_reserved"] is True
    assert state["negotiation_round"] <= policy.max_negotiation_rounds
    assert not state.get("security_block")
    assert state["final_amount_paise"] == state["approved_offer"]["final_amount_paise"]
```

Python

Raises `PaymentGateError` -> no Razorpay call.

Trace payload (guardrail decision)

```
{
  "decision_id": "550e8400-e29b-41d4-a716-446655440000",
  "outcome": "REJECTED",
  "policy_version": "2026.08.1",
  "rules": [
    {
      "rule_id": "RULE_MAX_DISCOUNT",
      "passed": false,
      "action": "CLAMP",
      "message": "Discount capped at 15%",
      "inputs": { "requested_pct": 40, "max_pct": 15 }
    }
  ],
  "offer_version": 3,
  "final_amount_paise": 424900
}
```

JSON

Trace UI: pass / clamp / reject per rule row.

Tests before merge

- Unit: each rule at 0%, 100%, negative edge
- Property: `final_amount_paise == subtotal - discount` always
- Integration: injection strings -> zero payment links
- Chaos: Razorpay 503 -> no duplicate charges

CI gate: fail if payment link created without APPROVED.

Default config (`config/policy_defaults.yaml`)

```

max_discount_pct: 15.0
min_margin_pct: 20.0
max_qty_per_line: 10
max_negotiation_rounds: 5
inventory_hold_ttl_seconds: 900
payment_link_expiry_seconds: 86400
anomaly_escalation_threshold: 0.85

```

YAML

Risk register (abridged)

ID	Risk	Control
R-01	Discount over cap	RULE_MAX_DISCOUNT
R-02	Injection cheap link	injection + gates
R-03	Hallucinated SKU	Postgres catalog only
R-04	Double link	idempotency key
R-05	Below margin	RULE_MIN_MARGIN
R-06	Oversell	FOR UPDATE holds
R-07	Forged webhook	HMAC + event_id
R-08	Audit edit	append-only trigger
R-09	LLM math	compute_totals only
R-10	Endless negotiate	5 round cap
R-11	Link spam	Redis rate limit
R-12	PII in trace	redact in TraceService

Response: detect -> halt money -> audit -> notify if PO -> fix.

Security protocols

ID	Rule
SP-01	<code>assert_payment_gates()</code> before every Razorpay call
SP-02	<code>audit_logs</code> INSERT only
SP-03	Webhook HMAC, reject stale events
SP-04	No secrets or margin in LLM context
SP-05	Human overrides logged; no margin override in v1
SP-06	<code>degradation_level</code> ≥ 3 blocks new links